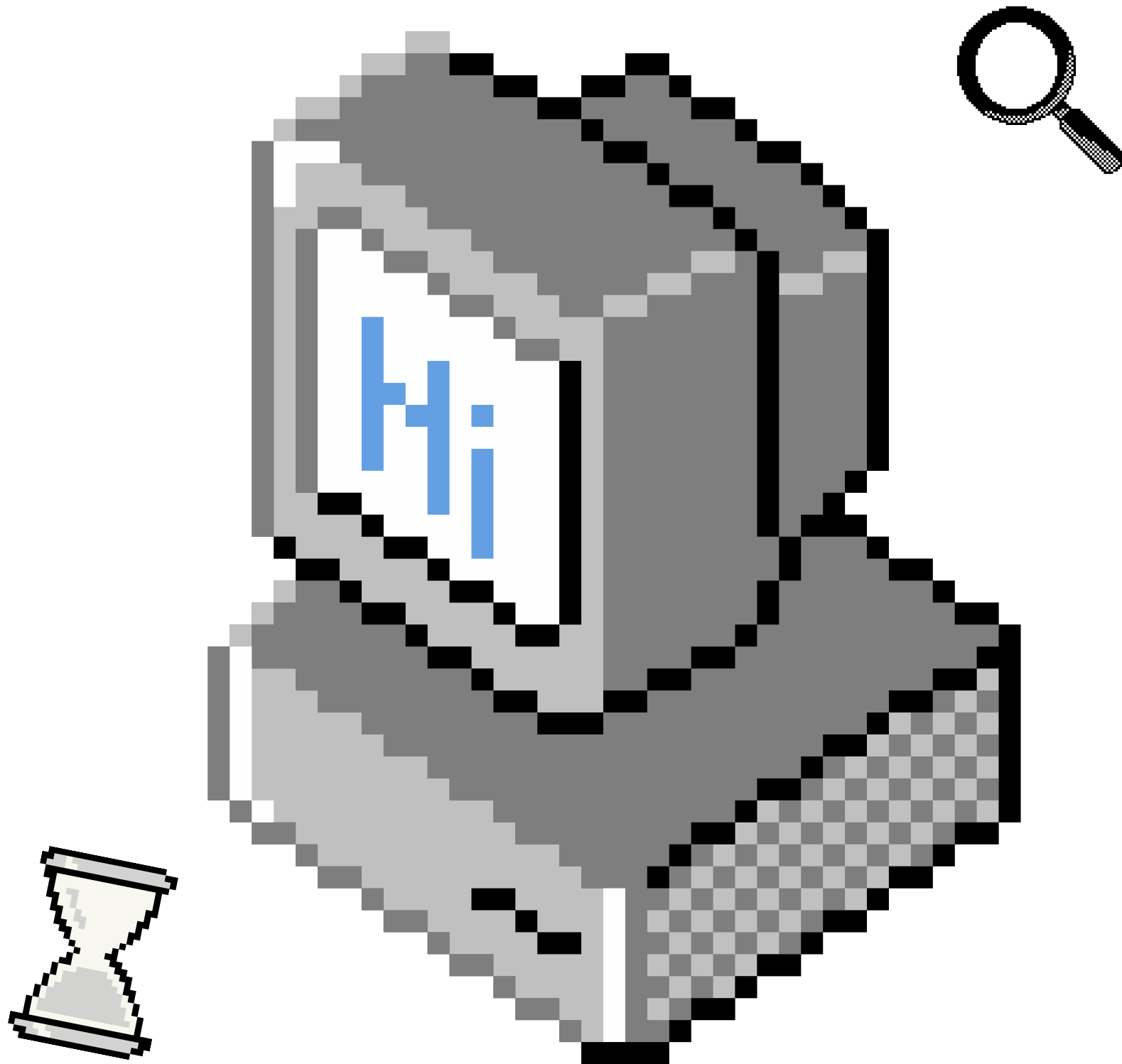


PIEDRA PAPEL O TIJERA

NOMBRE: PAULA CAMILA ARCOS HERRERA

MATERIA: LÓGICA DE PROGRAMACIÓN

CARRERA: INGENIERIA EN SOFTWARE

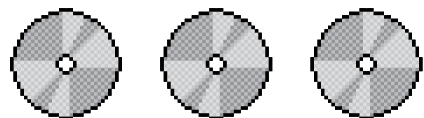


Fradel and Spies

INTRODUCCIÓN

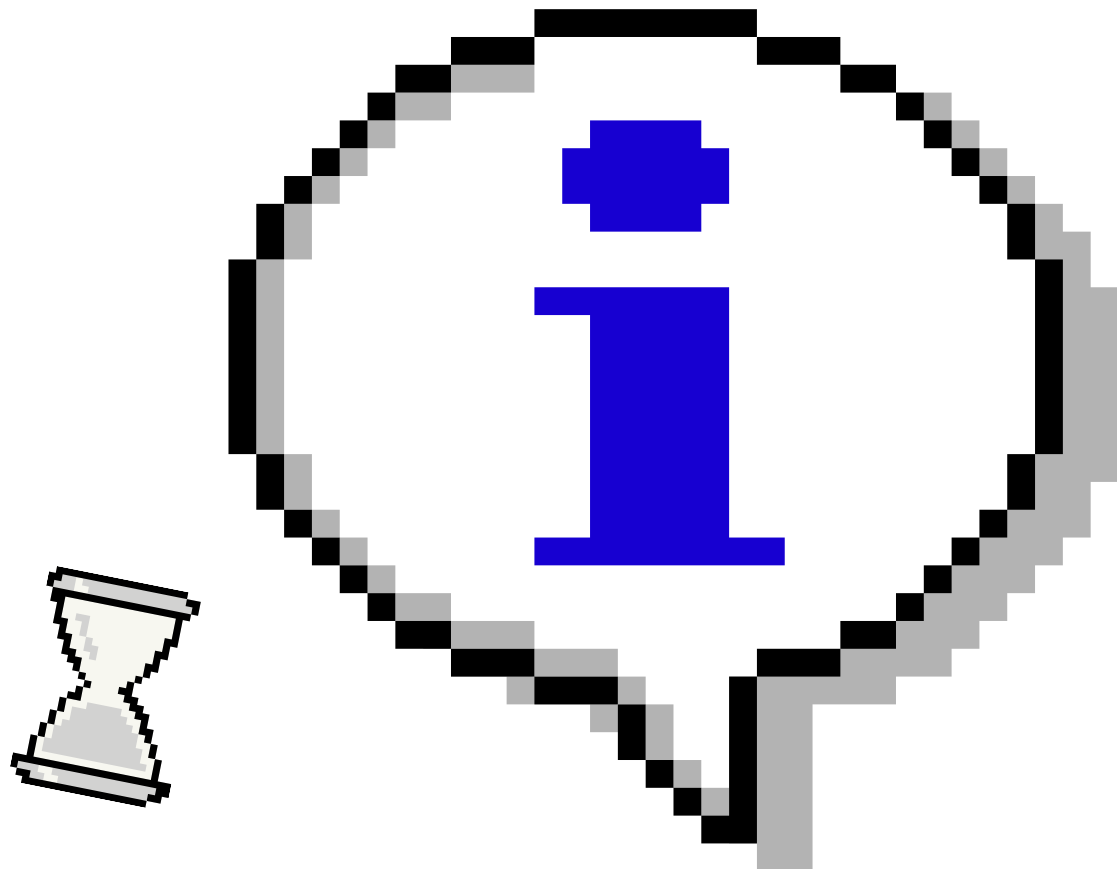
Este proyecto tiene como objetivo desarrollar una aplicación interactiva que permita al usuario jugar PIEDRA, PAPEL O TIJERA contra la computadora.

Para ello, se aplicaron conceptos básicos de programación como estructuras de control, funciones y manejo de datos.



OBJETIVO

Desarrollar un programa interactivo en Python que simule el juego “Piedra, Papel o Tijera”, aplicando principios de programación estructurada, incluyendo el uso de algoritmos, estructuras de control, manejo de datos y funciones, con el fin de consolidar los conocimientos adquiridos durante el curso.



OBJETIVOS ESPECÍFICOS

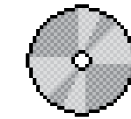
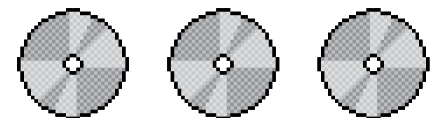
Analizar el problema planteado y diseñar una solución mediante la construcción de un algoritmo que represente la lógica del juego.

Implementar estructuras de control, tales como condicionales y bucles, para gestionar el flujo de ejecución del programa y la interacción con el usuario.

Utilizar estructuras de datos, específicamente listas, para almacenar y manipular las opciones del juego.

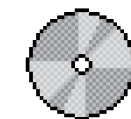
TECNOLOGÍAS UTILIZADAS

PROGRAMAS QUE PERMITIERON QUE EL PROGRAMA SE
LLEVARA A CABO



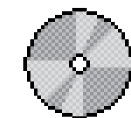
Python (Lenguaje)

El lenguaje principal del proyecto y el unico fue Python ya que permite un mejor entendimiento cuando se inicia en la programación



Visual Studio Code

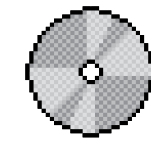
Fue el entorno de desarrollo, edicion y ejecucion del codigo del juego



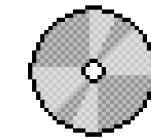
Github

Github permitio crear un repositorio donde se documenta cada parte del proyecto incluyendo las correcciones, explicacion, etc

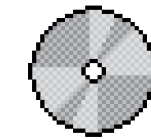
FUNCIONAMIENTO DEL JUEGO



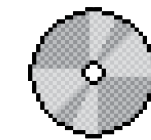
Usuario elige la opción



La computadora genera la opción



Se comparan los resultados

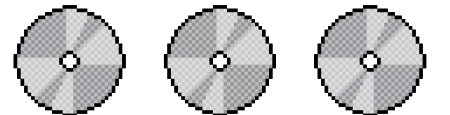


Se muestra el ganador





ESTRUCTURA DEL CÓDIGO



FUNCIONES PRINCIPALES

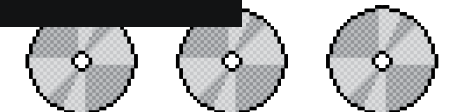


mostrar_titulo(): Presenta el encabezado del programa.

```
def mostrar_titulo():  
    """Muestra el título del juego"""  
    print("\n--- JUEGO PIEDRA, PAPEL O TIJERA ---")
```

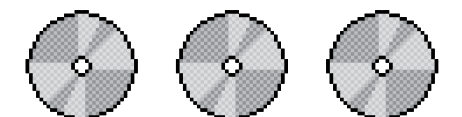
mostrar_opciones(): Muestra las opciones disponibles.

```
def mostrar_opciones():  
    """Muestra las opciones disponibles (uso de FOR)"""  
    print("Opciones disponibles:")  
    for i in range(len(opciones)): # UNIDAD 3: FOR  
        print(f"{i + 1}. {opciones[i]}")
```



obtener_eleccion_usuario(): Captura y valida la entrada del usuario.

```
def obtener_eleccion_usuario():  
    """  
    Solicita la elección del usuario.  
    Usa WHILE + IF + operadores lógicos  
    """  
    while True: # UNIDAD 3: WHILE  
        mostrar_opciones()  
        eleccion = input("Elige (piedra, papel o tijera): ").lower()  
  
        # UNIDAD 3: CONDICIONALES + OPERADORES LÓGICOS  
        if eleccion == "piedra" or eleccion == "papel" or eleccion == "tijera":  
            if eleccion in opciones: # UNIDAD 2: IN  
                return eleccion  
        else:  
            print("✗ Entrada inválida. Intenta nuevamente.\n")
```



obtener_eleccion_computadora(): Genera una elección aleatori

determinar_ganador(): Evalúa las condiciones del juego.

```
def mostrar_titulo():  
    """Muestra el título del juego"""  
    print("\n--- JUEGO PIEDRA, PAPEL O TIJERA ---")
```

```
def obtener_eleccion_computadora():  
    """Genera una elección aleatoria"""  
    return random.choice(opciones) # UNIDAD 2: MANEJO DE DATOS  
  
def determinar_ganador(usuario, computadora):  
    """Determina el resultado del juego (IF anidados)"""  
  
    # Empate  
    if usuario == computadora:  
        return "Empate"  
  
    # Condiciones para ganar  
    if (usuario == "piedra" and computadora == "tijera") or \  
        (usuario == "papel" and computadora == "piedra") or \  
        (usuario == "tijera" and computadora == "papel"):  
        return "Ganaste"  
  
    # Caso contrario  
    return "Perdiste"
```

jugar(): Controla el flujo principal del programa.

```
def jugar():  
    """  
    Función principal del programa.  
    Controla todo el flujo del juego.  
    """  
  
    # UNIDAD 3: BUCLE PRINCIPAL (WHILE)  
    while True:  
        mostrar_titulo()  
  
        # Entrada usuario  
        usuario = obtener_eleccion_usuario()  
  
        # Elección computadora  
        computadora = obtener_eleccion_computadora()  
  
        # Mostrar resultados  
        print(f"\nTú elegiste: {usuario}")  
        print(f"La computadora eligió: {computadora}")  
  
        # Determinar ganador  
        resultado = determinar_ganador(usuario, computadora)  
        print("Resultado:", resultado)  
  
        # Preguntar si desea continuar  
        continuar = input("\n¿Quieres jugar otra vez? (s/n): ").lower()  
  
        # Decisión final (como en tu diagrama)  
        if continuar != "s":  
            print("Fin del juego")  
            break
```


ESTRUCTURAS UTILIZADAS

- Estructuras condicionales (if, elif, else): Para la lógica de decisiones.
- Bucles (while, for): Para repetición y control del flujo.
- Listas: Para almacenar las opciones del juego.
- Funciones: Para dividir el problema en módulos.
- Operadores lógicos: Para evaluar condiciones complejas.



DIAGRAMA DE FLUJO



paulapcah-design/
Autonomo-2

piedra papel o tijera

1

Contributor

0

Issues

0

Stars

0

Forks

Autonomo-2/_Diagrama PIEDRA PAPEL O TIJERA.png at main • paulapcah-design/Autonomo-2

piedra papel o tijera. Contribute to paulapcah-design/Autonomo-2 development by creating an account on GitHub.

GitHub



paulapcah-design/
Autonomo-2

piedra papel o tijera

1

Contributor

0

Issues

0

Stars

0

Forks

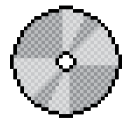
Autonomo-2/DIAGRAMA DE FLUJO FUNCIONES PIEDRA PAPEL O TIJERA.png at main • paulapcah-design/Autonomo-2

piedra papel o tijera. Contribute to paulapcah-design/Autonomo-2 development by creating an account on GitHub.

GitHub

DIAGRAMA DE FLUJO CON FUNCIONES

RESULTADOS



Correcto funcionamiento del código

```
--- JUEGO PIEDRA, PAPEL O TIJERA ---
```

```
Opciones disponibles:
```

```
1. piedra
```

```
2. papel
```

```
3. tijera
```

```
Elige (piedra, papel o tijera): piedra
```

```
Tú elegiste: piedra
```

```
La computadora eligió: papel
```

```
Resultado: Perdiste
```

```
¿Quieres jugar otra vez? (s/n): s
```

```
--- JUEGO PIEDRA, PAPEL O TIJERA ---
```

```
Opciones disponibles:
```

```
1. piedra
```

```
2. papel
```

```
3. tijera
```

```
Elige (piedra, papel o tijera): sjd
```

```
✗ Entrada inválida. Intenta nuevamente.
```

```
Opciones disponibles:
```

```
1. piedra
```

```
2. papel
```

```
3. tijera
```

```
Elige (piedra, papel o tijera): papel
```

```
Tú elegiste: papel
```

```
La computadora eligió: papel
```

```
Resultado: Empate
```

```
¿Quieres jugar otra vez? (s/n): n
```

```
Fin del juego
```

CONCLUSIONES

- Se logró integrar estructuras fundamentales de programación en un sistema funcional.
- La modularización mediante funciones mejoró la organización del código.
- El uso de validaciones incrementó la robustez del programa.
- Se evidenció la importancia de los diagramas de flujo en el diseño lógico.

